

9/9 SOLID 원칙 보고서

SOLID 원칙이란 객체 지향 프로그램 및 설계에서 코드의 유지보수성과 확장성을 높이기 위해 지켜야 할 5가지 소프트웨어 개발 원칙을 말한다. 이 **SOLID** 원칙을 가급적 지키는 것이 좋지만 실무에 맞게 유연하게 변경하는 능력도 필요하다.

Single Responsibility Principle (단일 책임 원칙)

: 클래스는 하나의 책임만 가지고 있어야 한다.

SRP Noncompliant example을 보면 `UserService`라는 클래스에 `saveUser`(유저 정보 저장), `sendWelcomeMail`(환영 메세지 전송), `logUserActivity`(유저 로그 기록)하는 기록까지 수행한다. 이런식으로 코드를 작성하면 각 클래스의 코드가 복잡해지고 이후의 유지보수가 어려워지는 문제가 발생한다. 그 이유는 클래스의 책임이 많아지면 각 클래스를 수정할 이유가 많아지고 그렇게 되면 클래스가 다른 클래스에 잘못된 영향을 미칠 가능성이 올라가게 되기 때문이다. 따라서 여러 책임이 한곳에 많이 있으면 테스트와 리팩토링도 까다로워 진다.

SRP Noncompliant example 출력

```
User saved to database: Hong Gildong
Welcome email sent to: hong@example.com
Logging activity for user: Hong Gildong
```

SRP compliant example 출력

```
User saved to database: Hong Gildong
Welcome email sent to: hong@example.com
Logging activity for user: Hong Gildong
```

출력에서는 차이가 들어나지 않지만 유지 보수적인 측면에서 **SRP**를 지키는 것이 좋다.

Open/Closed Principle (개방/폐쇄 원칙)

: 클래스가 확장에는 열려 있어야 하고, 변경에는 닫혀 있어야 한다.

OCP noncompliant example을 보면 문서를 생성하는 함수에 **PDF**, **HTML**을 생성할수 있는 기능이 있다. 지금은 각 기능이 한 줄로 편향되어 있어 파악하고 이해하기 쉽지만 나중에 코드가 길어지거나 **PDF**, **HTML**이 아닌 다른 형식의 파일을 생성하는 기능을 만들려고 할 때에는 클래스 안의 함수를 계속 수정해야할 것이다. 그러면 새로운 케이스들이 추가되면서 이 함수는 계속해서 커지고 복잡해지면서 유지보수하기 어려워질 것이다. 문제를 해결하기 위해 사용되는 원칙이 **OCP**이다. 먼저 **compliant example**에서처럼 문서를 생성하는 기능은 껍데기를 인터페이스로 만들어 두고 해당 인터페이스를 적용한 클래스들이 이 함수를 오버라이해서

각자의 역할을 하는것이다. 이렇게 되면 새로운 형식 문서를 생성해야될 때 다른 클래스만들면 되는 것이다. 이렇게 되면 기존 코드를 수정하는 과정에서 발생하는 문제로부터 자유로운 설계를 할 수 있다.

OCP1 출력 :

```
Generating PDF report...
Generating HTML report...
```

OCP2 출력 :

```
Generating PDF report...
Generating HTML report...
Generating XML report...
```

XML파일 생성 기능을 추가할 때 새로운 클래스를 만들어 추가하면 코드 수정하면서 발생하는 문제를 예방할 수 있다.

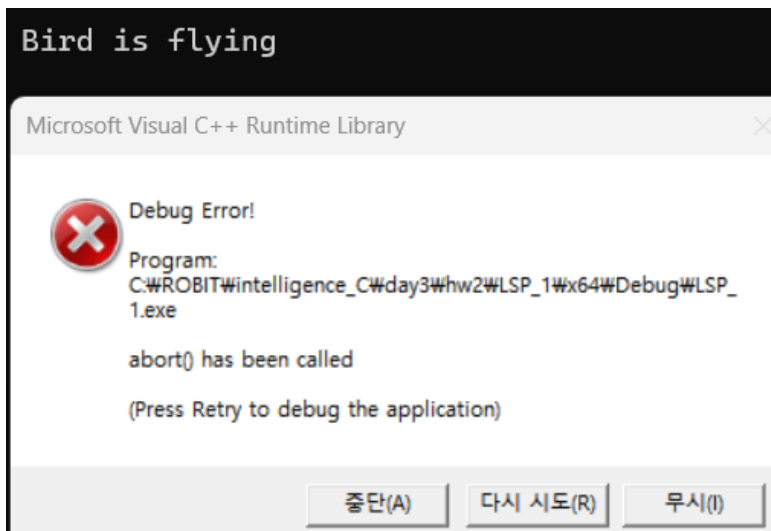
Liskow Substitution Principle (리스코프 치환 원칙)

: 자식 클래스는 언제나 부모 클래스를 대체할 수 있어야 한다.

LSP는 부모 클래스의 객체가 들어갈 자리에 자식 클래스의 객체가 들어가도 적어도 부모 클래스가 기존에 하던 일에는 지장이 없어야 한다는 것이다.

LSP_1을 보면 penguin 클래스는 BIRD의 자식 클래스로 fly()라는 기능을 물려받지만 penguin은 날지 못하기 때문에 부모 클래스에서는 fly()가 정상작동하지만 자식 클래스에서는 예외를 발생시킨다. 그래서 이는 잘못된 설계가 되는 것이다.

LSP_1 출력



이를 해결하려면 LSP_2처럼 fly함수를 최고 조상으로부터 물려받는 함수가 아니라 독립된 인터페이스로 분리해야한다.그리고 나서 bird 클래스중 날 수 있는 새에만 fly()를 적용하면 된다. 이렇게 되면 LSP를 위반하지 않을 수 있다.

LSP_2 출력

```
Bird is eating  
Sparrow is flying  
Bird is eating
```

다른 예시인 LSP_3을 보면 정사각형 클래스는 직사각형 클래스를 상속받고 있다. 직사각형에서는 너비와 높이를 따로 설정할 수 있지만, 정사각형에서는 너비를 설정하면 높이가, 높이를 설정하면 너비가 자동으로 설정되기 때문에 의도와 다르게 동작할 수 있다.

LSP_3 출력

```
Area: 20  
Area: 16
```

```
rectangle.setWidth(5);  
rectangle.setHeight(4);
```

을 했을 때 정사각형의 한 변이 4인것으로 덮어 씌어져서 16이라는 결과가 나오게 된다. 이를 해결하기 위해서도 부모자식 관계를 제거하고 **shape**이라는 별개의 클래스를 만드는것이 바람직하다.

LSP_4 출력

```
Area: 20  
Area: 16  
Area: 20  
Area: 25
```

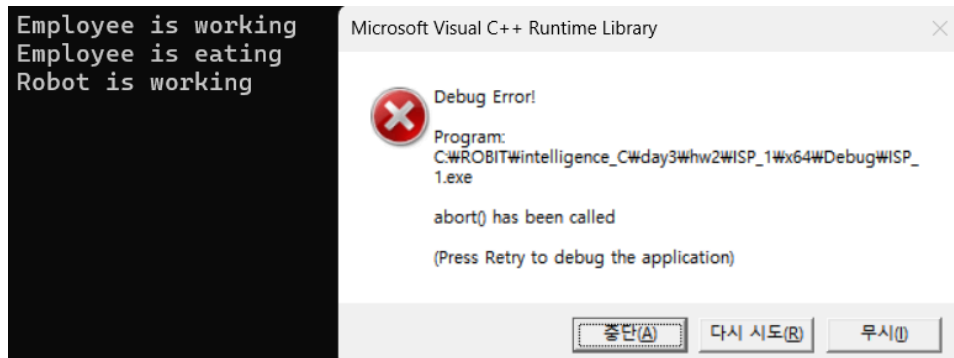
Shape이라는 클래스를 나눴을 때 정사각형 **area**가 25로 출력되는 것을 볼 수 있다.

Interface Segregation Principle (인터페이스 분리 원칙)

:클래스는 자신이 사용하지 않을 함수를 구현하도록 강요받지 말아야 한다.

Interface **worker**를 보면 **worker**는 **work()**와 **eat()**두개의 기능을 수행 한다. 그리고 **robott**, **employee**는 이 **work**를 적용받게 된다. 이때 노동자는 먹기도 하고 일도 하지만 **robot**은 먹지 않고 일만 한다. 이렇게 되면 로봇에게는 **eat()**이 잉여 함수가 되는 것이다.

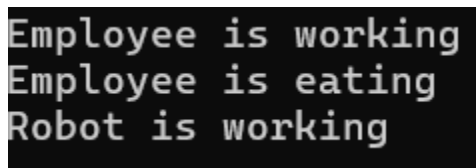
ISP_1 출력



로봇은 먹는 일을 하지 않기에 오류가 발생할 수 있다.

이런일을 방지하기 위해 인터페이스를 **workable**, **eatable**로 나누고 인간 직원은 두개의 인터페이스를 적용받고, 로봇 직원은 **workable**만 적용 받도록 수정하는 것이 좋다.

ISP_2

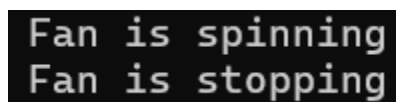


Dependency Inversion Principle (의존성 역전 법칙)

: 고수준 모듈이 저수준 모듈에 의존해서는 안된다

DIP_1 을 보면 **Switch** 클래스 내부에 **Fan** 클래스의 인스턴스 속성을 포함하고 스위치를 켜고 끄는 함수를 실행하면 선풍기가 동작하거나 멈추도록 되어 있다. 이때 DIP 위배가 일어난다. 즉 스위치를 켜고 끄는 것은 저수준 클래스에 있는 저수준 모듈에 선풍기를 동작시키 것인 고수준 클래스에 있는 고수준 모듈이 의존하게 되어 있다는 것이다. 코드를 보면 **spin**, **stop** 함수들을 비롯한 선풍기 클래스의 코드들이 스위치 클래스 내부에서도 그대로 사용되는 것을 볼 수 있다. 이렇게 되면 **fan**의 함수를 변경하게 된다면 **switch** 클래스의 실행부도 이에 따라 수정해야되고 이렇게 되면 유지보수가 어려워지게 된다. 또 이런 구조에서는 스위치가 선풍기만 다룰수 있게 되어 확장성도 떨어진다.

DIP_1 출력



DIP_2에서는 이를 해결하고자 **Switchable**이라는 인터페이스를 따로 만들고 선풍기가 **Switchable**이라는 카테고리에 속하게 만든다. 이렇게 되면 **swith**와 **fan**이 직접 연결되지 않고

switchable이란 인터페이스를 거쳐 소통하게 되면 **fan**의 함수가 수정되어도 **swithc**의 클래스에 영향을 주지 않게 된다.

DIP_2 출력

```
Fan is spinning  
Fan is stopping
```

출력 결과에는 변화가 없지만 이 원칙도 유지보수하기 편하게 하는 원칙이다.